

# OneKhusa Python Integration Reference

## Hosted Checkout with FastAPI and Socket.io

Onekhusa Team

May 2026

## 1 Introduction

This documentation serves as a professional reference for developers implementing the OneKhusa Hosted Checkout flow using Python. The integration leverages **FastAPI** for high-performance asynchronous routing and **python-socketio** to bridge the communication gap between the backend webhook and the client-side dashboard.

## 2 Project Directory Structure

The application is organized into a modular structure to separate API logic from the server implementation.

```
onekhusa-python-integration/
├── .env .....Environment configuration
├── requirements.txt .....Python dependencies
├── app/
│   ├── main.py .....FastAPI Entry Point & Socket setup
│   └── services/
│       └── onekhusa_service.py .....Checkout Initiation Logic
└── public/
    └── index.html .....Dashboard UI & Verification
```

## 3 The Brain: Environment Configuration (.env)

The `.env` file stores credentials and mapping for the NGrok public tunnel used during development.

```
1 ONEKHUSA_API_KEY=sandbox_vDdLumyf...
2 ONEKHUSA_API_SECRET=_fefXaN9LGorR...
3 ONEKHUSA_ORG_ID=0BBNREQ33RSX
4 ONEKHUSA_MERCHANT_NUMBER=79619974
5
6 ONEKHUSA_CHECKOUT_URL=https://api.onekhusa.com/sandbox/v1/checkout/rtp/initiate
7 PUBLIC_CALLBACK_URL=https://zena-unjudgeable-renita.ngrok-free.dev
8 PORT=8000
```

Listing 1: .env Configuration

## 4 Core Service Implementation

The service layer handles the HTTP `POST` request to OneKhusa. Note the use of `apiSecret` (camel-Case) required by the Hosted Checkout endpoint.

```
1 import requests
2 import uuid
3
4 class OneKhusaService:
```

```

5     def initiate_hosted_checkout(self, amount, reference, description):
6         payload = {
7             "authentication": { "apiKey": self.api_key, "apiSecret": self.api_secret
8         },
9             "merchant": { "organisationId": self.org_id, "merchantAccountNumber":
10         self.merchant_no },
11             "payment": { "sourceReferenceNumber": reference, "description":
12         description, "amount": amount },
13             "route": {
14                 "successRedirectUrl": f"{self.callback_base}?ref={reference}",
15                 "failureRedirectUrl": f"{self.callback_base}?ref={reference}",
16                 "callbackApiUrl": f"{self.callback_base}/webhooks/payments"
17             }
18         }
19         headers = { "X-Idempotency-Key": f"PY-CHK-{reference}-{str(uuid.uuid4())[:8]}"
20     }
21     response = requests.post(self.checkout_url, json=payload, headers=headers)
22     response.raise_for_status()
23     return response.json()

```

Listing 2: app/services/onekhusa\_service.py

## 5 FastAPI Server and Webhook Listener

The main controller manages an in-memory `ticket_tracker` and uses `sio.emit` to notify the dashboard of successful payments.

```
1 @app.post("/webhooks/payments")
2 async def handle_webhook(request: Request):
3     payload = await request.json()
4     metadata = payload.get("metaData", {})
5
6     # Handle TitleCase metadata keys from Sandbox
7     my_ref = metadata.get("ReferenceNumber") or payload.get("sourceReferenceNumber")
8
9     if payload.get("responseCode") == "S100" or payload.get("transactionStatusCode")
    == "S":
10         ticket_tracker[my_ref] = "Paid"
11         # Broadcast real-time update to UI
12         await sio.emit('webhook_received', {'reference': my_ref, 'status': 'Paid'})
13
14     return PlainTextResponse("acknowledged")
```

Listing 3: app/main.py Webhook Handler

## 6 Conclusion

Python's concise syntax and FastAPI's native support for asynchronous operations make this integration robust and scalable. By utilizing a unified redirection strategy and real-time socket communication, the system provides a seamless experience for both mobile and web users during the payment lifecycle.